

School of Management Sciences, Lucknow

Data Structures

From Arrays to Linked Lists, Stacks, Queues & Complexity Analysis

Agenda

What we will cover in this session

01

Introduction to Data Structures

Fundamentals, types and importance of data structures

02

Arrays & Strings

1D and multi-dimensional arrays, string operations and algorithms

03

Linked Lists

Singly, doubly and circular linked lists

04

Stacks & Queues

LIFO, FIFO, core operations and practical applications

05

Complexity Analysis

Time complexity, space complexity and Big-O notation

06

Hands-on & Key Takeaways

Examples, problem solving and recap

Introduction to Data Structures

Learn the fundamentals of data organisation and why choosing the right data structure matters.

What is a Data Structure?

A **data structure** is a systematic way of organising, storing, and managing data so that it can be accessed and modified efficiently.

Why Do We Need Data Structures?

- Organise large amounts of data
- Make data access faster
- Enable efficient insertion and deletion
- Improve program performance
- Provide suitable structures for different problems

Examples

Array → Store elements sequentially

Linked List → Dynamic collection of elements

Stack → Last-In-First-Out

Queue → First-In-First-Out



Key Highlight:

Choosing the right data structure can significantly improve the efficiency of an algorithm.

Arrays

Storing related elements in contiguous memory locations

What is an Array?

An **array** is a collection of elements of the same type stored in a sequential manner.

Key Characteristics

- Elements are accessed using an index
- Indexing generally starts from 0
- Elements are stored sequentially
- Direct access is possible
- Size may be fixed depending on the language

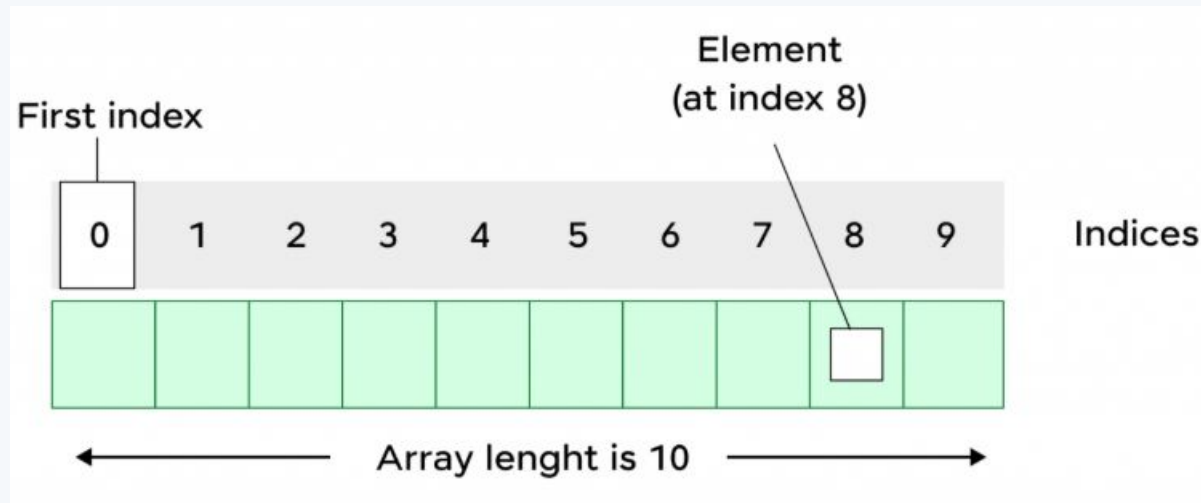
Example

marks = [85, 90, 78, 92, 88]

print(marks[2])

Output:

78



Array Operations

Common operations:

Declaration in Python

Using list: `arr = [10, 20, 30, 40, 50]`

Using array module: `import array; arr = array.array('i', [1,2,3])`

Core Operations

- Access: `arr[i]` – $O(1)$ direct index calculation
- Search: scan left to right – $O(n)$ linear scan
- Insertion at end: $O(1)$ if space available | at index i : $O(n)$ shift
- Deletion at index i : shift left $O(n)$ | at end: $O(1)$

One-Dimensional Arrays

A one-dimensional array stores elements along a **single dimension**.

[10, 20, 30, 40, 50]

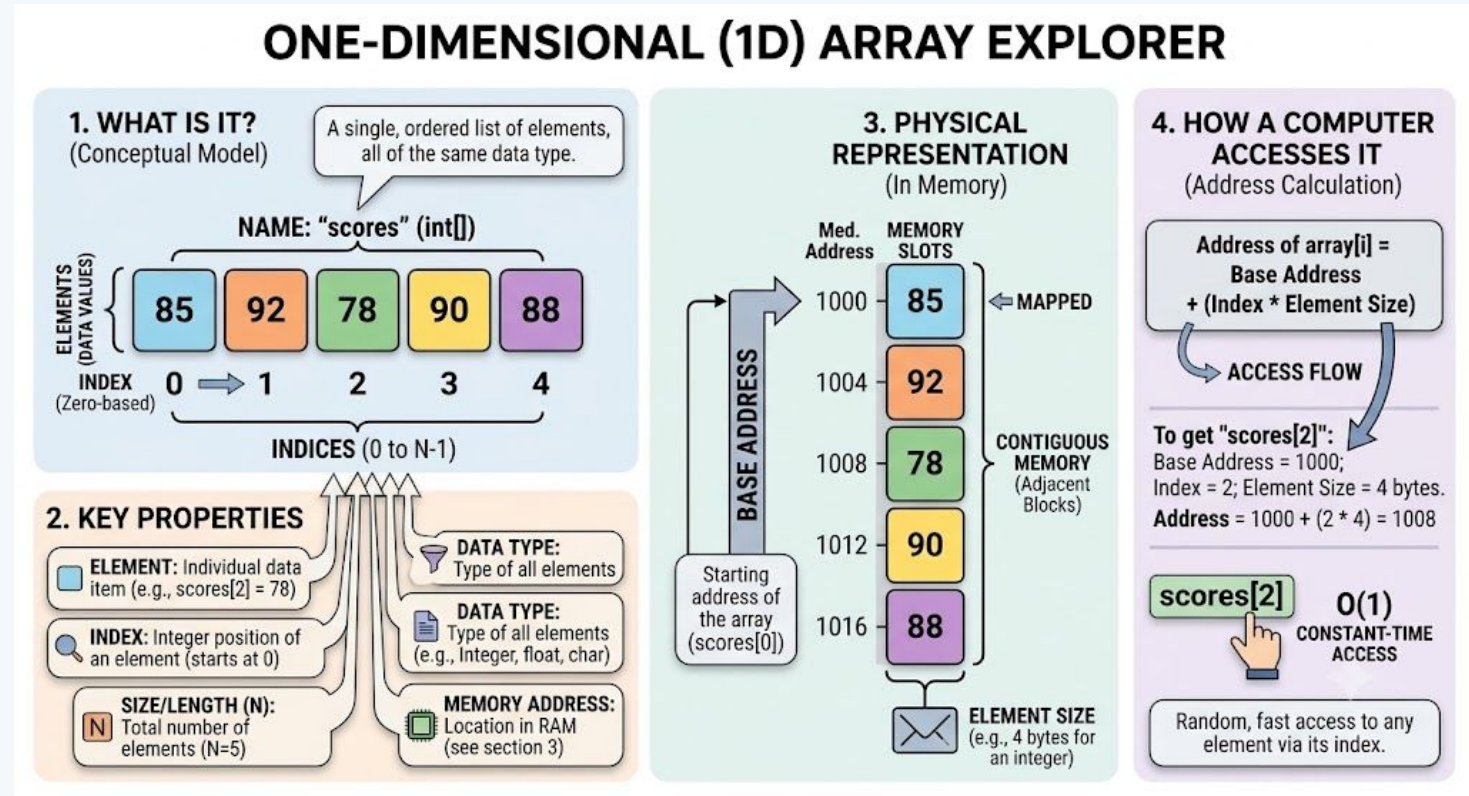
Example:

```
numbers = [10, 20, 30, 40, 50]
```

```
for i in range(len(numbers)):
    print(numbers[i])
```

Applications

- Student marks
- Employee salaries
- Product prices
- Sensor readings
- Searching and sorting datasets



Multi-Dimensional Arrays

A **multi-dimensional array** contains data organised across **multiple dimensions**.

Example:

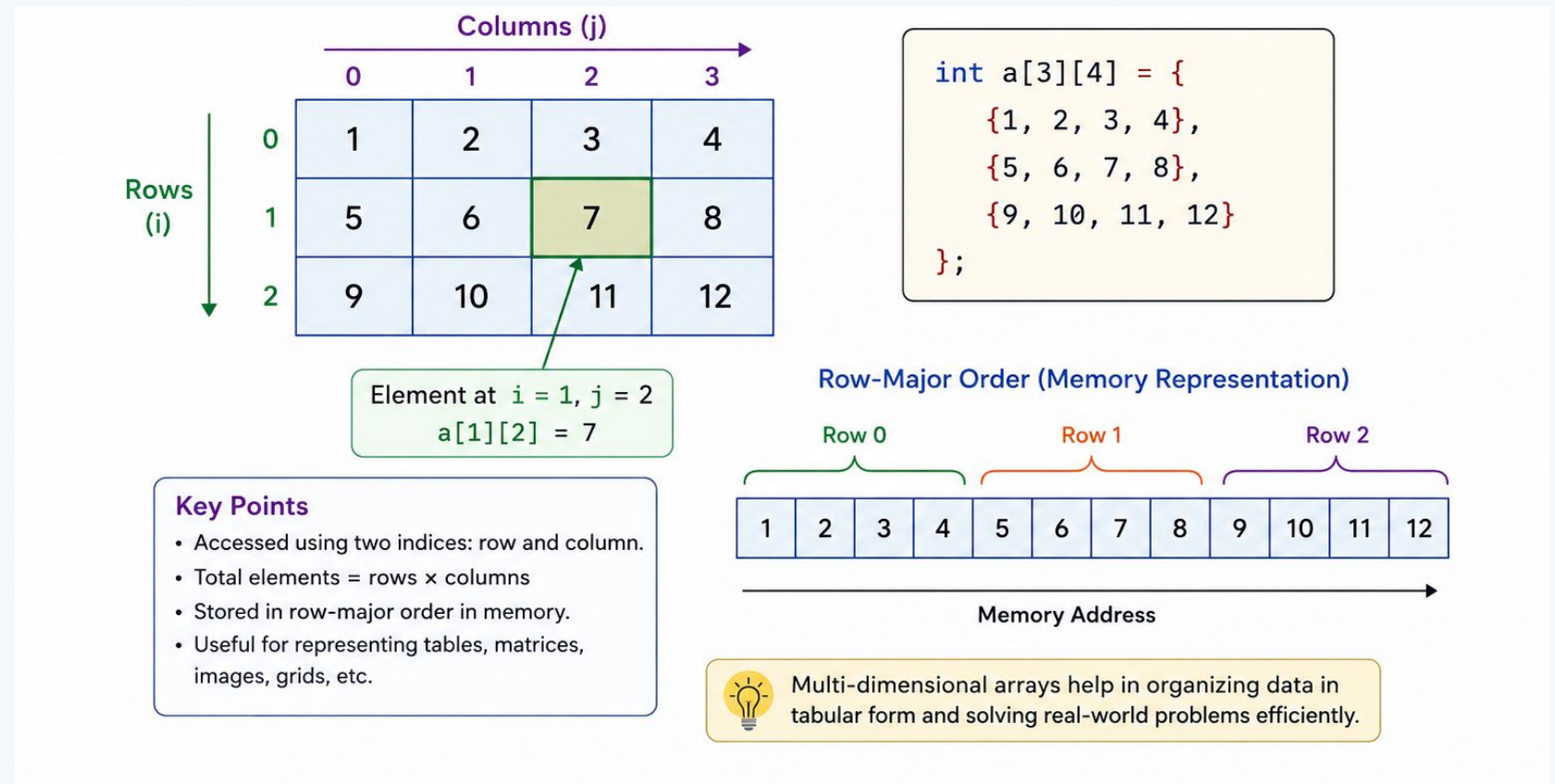
```
matrix = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]
```

```
print(matrix[1][2])
```

Output: 60

Applications

- Matrices
- Image processing
- Game boards
- Tables
- Scientific computing



STRINGS

A **string** is a sequence of characters.

Declaration in Python

Using single quotes:

```
text = 'Hello World'
```

Using double quotes:

```
text = "Hello World"
```

Using triple quotes:

```
text = """Hello World"""
```

Core Operations

- **Access:** `text[i]` – $O(1)$ direct index access
- **Search:** `text.find("World")` / scan left to right – $O(n)$
- **Concatenation:** `text1 + text2` – $O(n + m)$ creates a new string
- **Slicing:** `text[start:end]` – $O(k)$ where k is the slice length
- **Length:** `len(text)` – $O(1)$

Other Useful Operations

- `upper()`
- `lower()`
- `replace()`
- `split()`
- `strip()`
- `find()`

String Algorithms

String algorithms are used to **search, compare, process, and manipulate sequences of characters.**

Common Algorithms

Naive Pattern Matching

- Checks the pattern at every possible position
- Simple to understand
- Can become inefficient for large strings

KMP Algorithm

- Knuth-Morris-Pratt
- Avoids unnecessary comparisons
- Uses the **LPS (Longest Prefix Suffix)** concept

Other Examples

- Rabin-Karp
- Z Algorithm
- String hashing

Applications

- Search engines
- Text editors
- DNA sequence analysis
- Plagiarism detection
- Pattern matching

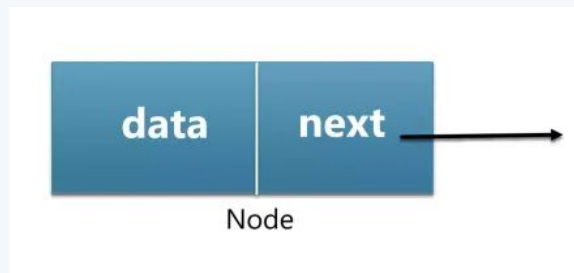
LINKED LISTS

Dynamic data structures built using connected nodes

What is a Linked List?

A linked list is a collection of **nodes**, where each node contains:

A linked list is a dynamic data structure where each element (node) stores data and a pointer to the next node. Nodes are stored in non-contiguous memory.



Example:

Head



[10 | •] → [20 | •] → [30 | •] → NULL

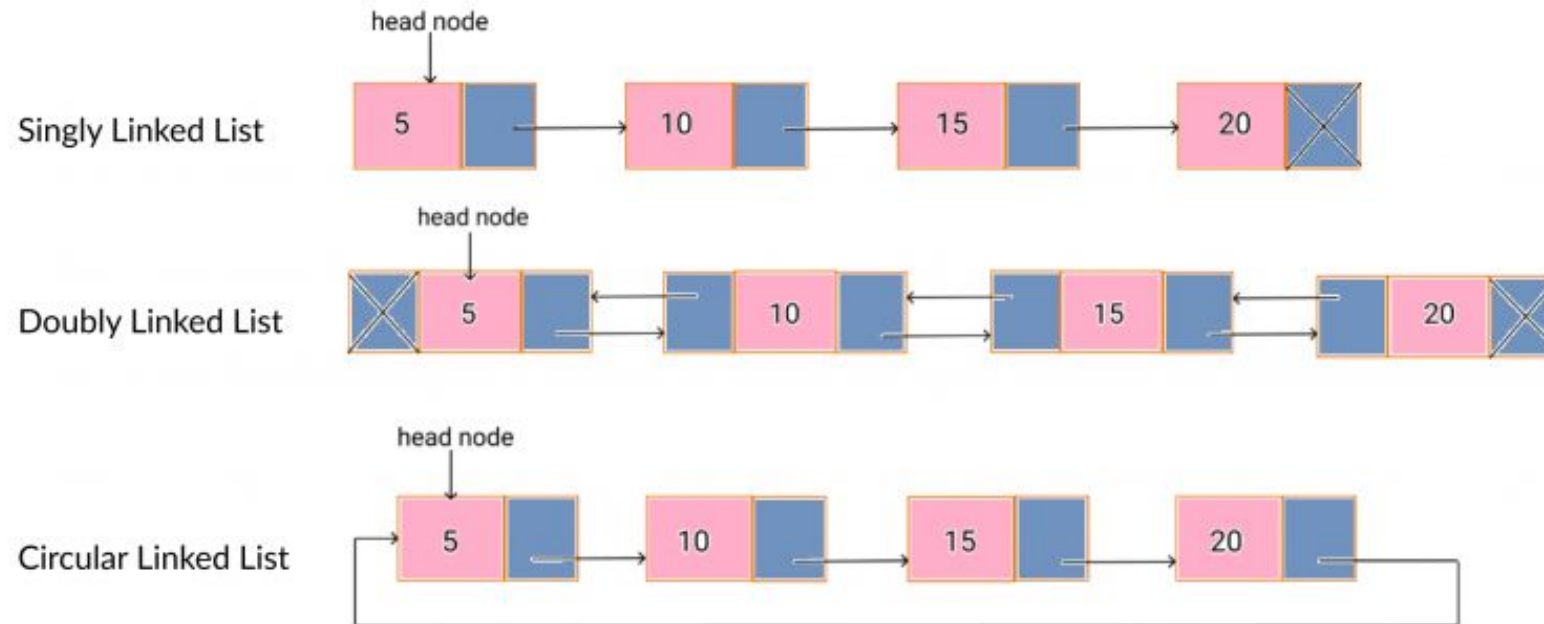
Key Features

- Dynamic size
- Nodes are connected using links
- Efficient insertion/deletion at known positions
- No requirement for contiguous memory
- Sequential access

Limitation

Direct random access is not available like arrays.

Types of Linked Lists



Singly Linked List

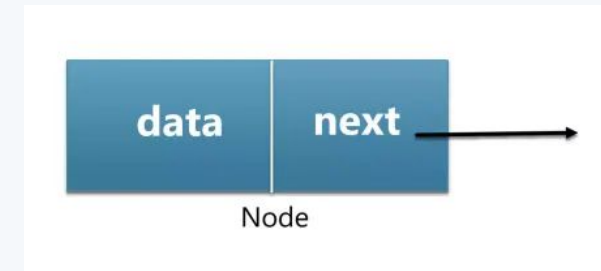
Each node points to the next node.

Node Structure

```
class Node:
    def __init__(self, data):
        self.data = data;
        self.next = None
```

Singly Linked List Operations

- Insert at Head: create node, point to current head, update head – $O(1)$
- Insert at Tail: traverse to last node, set `last.next = new_node` – $O(n)$
- Delete by Value: find node before target, redirect pointer – $O(n)$
- Search: traverse from head until match – $O(n)$



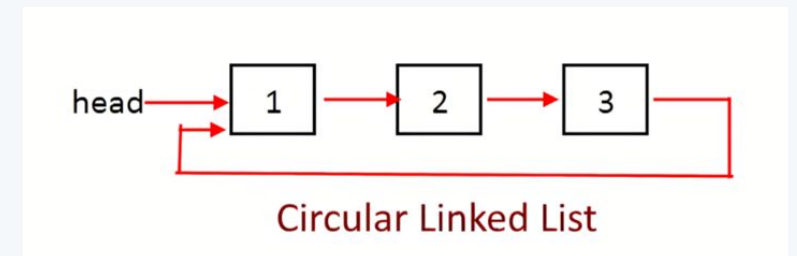
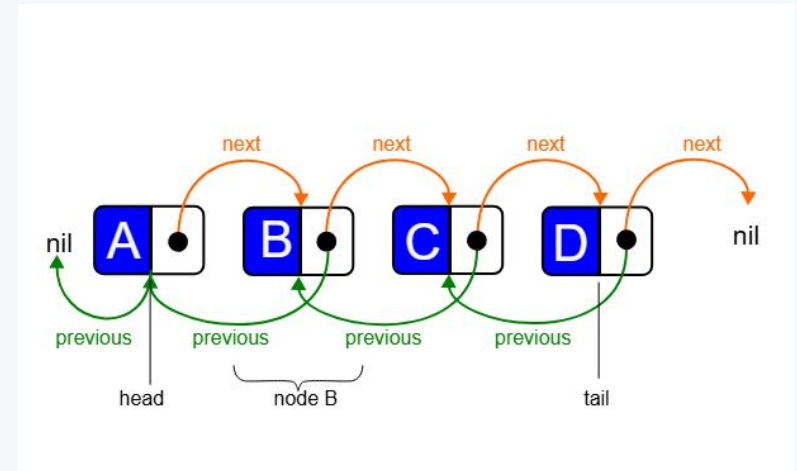
Doubly and Circular Linked Lists

Each node points to the next node.

Doubly Linked List

class Node: data, next, prev – each node has TWO pointers

- Traverse in both directions: forward via .next, backward via .prev
- Insert/Delete at known node: $O(1)$ – no need to find predecessor
- Used in: browser history, LRU Cache, music playlist (back/forward)
- Trade-off: extra memory per node for the prev pointer



Linked List Operations

1

Create

Create nodes

2

Traverse

Visit each node
sequentially

3

Insert

Add a new node

4

Delete

Remove an existing node

5

Search

Find a specific value

Stacks & Queues

Linear data structures based on controlled access principles

What is a Stack?

A stack is a linear ADT that follows LIFO (Last In, First Out) – the element inserted last is removed first, like a stack of books.

Core Properties

- Only the TOP element is accessible at any time
- All insertions (push) and deletions (pop) happen at the same end
- Peek (top): view top without removing – $O(1)$

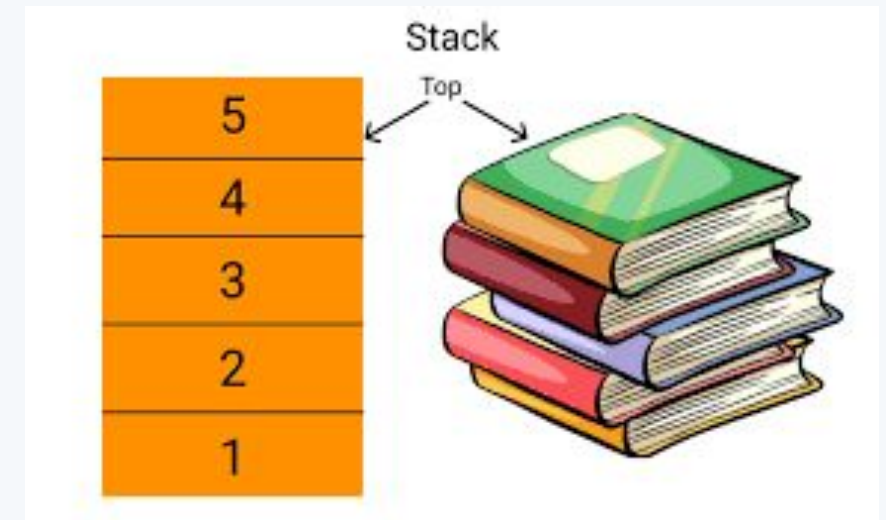
Main Operations

Push → Add an element

Pop → Remove the top element

Peek/Top → View the top element

isEmpty() → Check whether the stack is empty



Stack Operations & Applications

Linked List-Based Stack

- push: create node, set node.next = head, head = node – $O(1)$
- pop: save head.data, head = head.next, return saved – $O(1)$

Stack Overflow and Underflow

- Overflow: push onto a full array-based stack – raise Exception
- Underflow: pop from an empty stack – raise IndexError

Complexity Summary

- Push: $O(1)$ | Pop: $O(1)$ | Peek: $O(1)$ | Search: $O(n)$
- Space: $O(n)$ for n elements

Expression Evaluation and Conversion

- Infix to Postfix: use operator stack with precedence rules
- Postfix evaluation: push operands; on operator, pop two and compute
- Example: $3\ 4\ +\ 5\ *$ → push 3, push 4, pop+add=7, push 7, push 5, pop*=35

Parenthesis Matching / Balanced Brackets

- Push '(', '[', '{' on encountering; on ')' pop and check if it matches
- Stack empty at end → balanced | Mismatch or non-empty → invalid

What is a Queue?

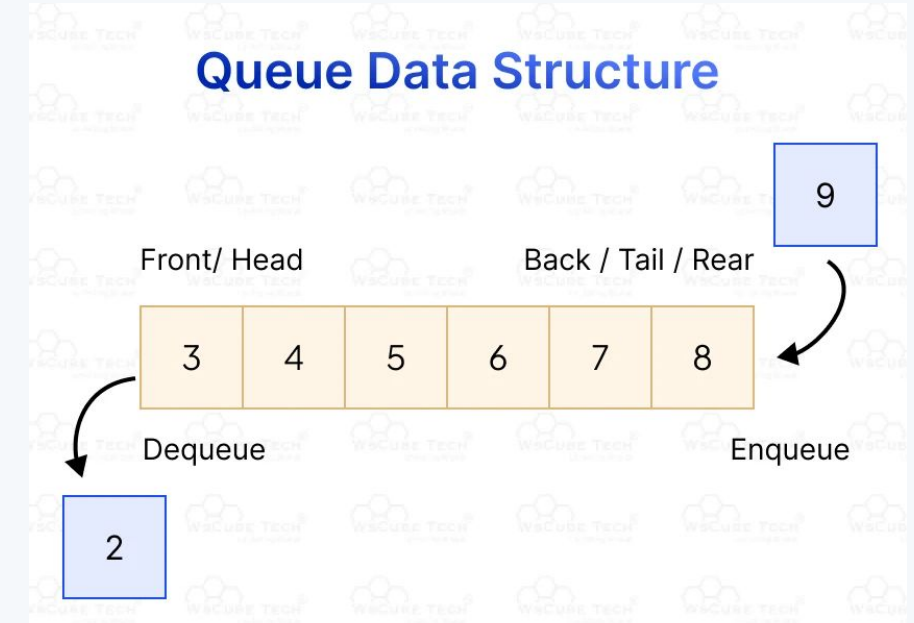
A queue is a linear ADT that follows FIFO (First In, First Out) – the element inserted first is the first to be removed, like a ticket counter line.

Core Properties

- Two ends: REAR (insertion end) and FRONT (removal end)
- Enqueue: add to REAR | Dequeue: remove from FRONT
- Peek/Front: view front element without removing – $O(1)$

Main Operations

- **Enqueue** → Add element
- **Dequeue** → Remove element
- **Front/Ppeek** → View first element
- **isEmpty()** → Check whether queue is empty



Types & Applications

Types of Queues

- Simple Queue: basic FIFO with fixed capacity
- Circular Queue: rear wraps to front to reuse empty slots
- Priority Queue: element with highest priority dequeued first
- Deque (Double-Ended Queue): insert/delete at both ends

Python Implementation

```
from collections import deque | q = deque()
q.append(x) # enqueue at rear | q.popleft() # dequeue from front
```

CPU and Process Scheduling

- FCFS (First Come First Served): processes in a simple queue
- Round Robin: circular queue; each process gets equal time slice
- Priority Queue: higher-priority processes preempt lower ones

Breadth-First Search (BFS)

- Queue holds nodes to visit: enqueue start node, dequeue and explore neighbors
- Guarantees shortest path in unweighted graphs
- Applications: shortest path, level-order tree traversal, social network proximity.

COMPLEXITY ANALYSIS

Why Analyze Algorithms?

The Problem

Two algorithms can produce the same result but have very different performance.

Algorithm A

Input: 1,000,000 elements

Time: Very high

Algorithm B

Input: 1,000,000 elements

Time: Much lower

Complexity Analysis Helps Us Understand

- How execution time grows
- How memory requirements grow
- Scalability
- Efficiency
- Algorithm selection

Two Important Measures

Time Complexity

How much computational time an algorithm requires.

Space Complexity

How much memory an algorithm requires.

Time Complexity

What is Time Complexity?

Time complexity describes how the **running time of an algorithm grows as the input size increases**.

It does not necessarily represent actual seconds.

Example

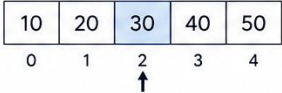
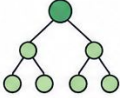
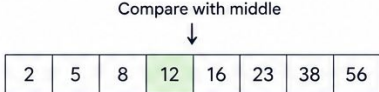
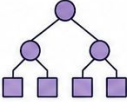
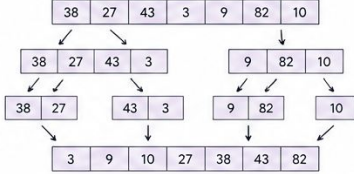
```
for i in range(n):
    print(i)
```

The loop executes approximately n times.

Time Complexity

$O(n)$

Common Time Complexities

Complexity	Example
$O(1)$ Constant Time 	Array access Accessing an element in an array using an index. 
$O(\log n)$ Logarithmic Time 	Binary Search Searching in a sorted array by repeatedly dividing the search space. 
$O(n)$ Linear Time 	Linear Search Checking each element one by one until the target is found. 
$O(n \log n)$ Linearithmic Time 	Merge Sort Divide the array into halves, sort them and merge. 
$O(n^2)$ Quadratic Time 	Bubble Sort Comparing each element with every other element. 

Space Complexity

What is Space Complexity?

Space complexity measures the **additional memory required by an algorithm as the input size grows**.

Example

```
def create_array(n):
    arr = []

    for i in range(n):
        arr.append(i)

    return arr
```

The algorithm creates storage proportional to n .

Space Complexity

$O(n)$

COMMON SPACE COMPLEXITIES

Space Complexity	Example
 $O(1)$ Constant Space	Simple variables, array access
 $O(\log n)$ Logarithmic Space	Recursion in binary search, balanced trees
 $O(n)$ Linear Space	Extra array of size n, recursion depth n
 $O(n \log n)$ Linearithmic Space	Merge sort, some divide and conquer algorithms
 $O(n^2)$ Quadratic Space	2D arrays, adjacency matrix

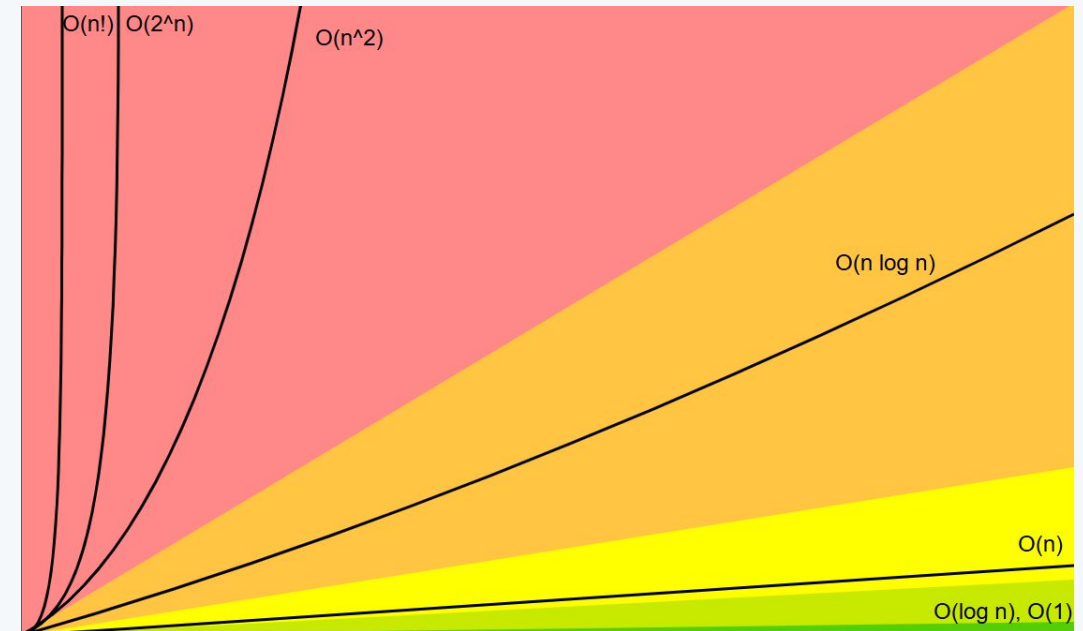
Big-O Notation

What is Big O Notation?

Big O describes how an algorithm's runtime or space grows relative to input size n as n approaches infinity – measuring worst-case asymptotic complexity.

Why It Matters

- An $O(n^2)$ algorithm on $n=10^6$ inputs makes 10^{12} operations – impractical
- Helps compare algorithms independently of hardware



Complexity Comparison

Big-O	Growth	Example	Visual Growth
$O(1)$	Constant	Array access	
$O(\log n)$	Very slow growth	Binary Search	
$O(n)$	Linear	Linear Search	
$O(n \log n)$	Moderate	Merge Sort	
$O(n^2)$	Fast growth	Bubble Sort	
$O(2^n)$	Very fast	Some recursive solutions	

Data Structure	Access	Search	Insert	Delete
 Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
 Singly Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
 Stack	$O(n)^{**}$	$O(n)$	$O(1)$	$O(1)$
 Queue	$O(n)^{**}$	$O(n)$	$O(1)$	$O(1)$

* Insertion / Deletion at the beginning (head).

** Access to middle elements is $O(n)$.

APPLICATION & RECAP

Choosing the Right Data Structure

1

Problem

Create nodes

2

Identify Operations

What do we need frequently?

3

Select Data Structure

Array / Linked List / Stack / Queue

4

Analyze Complexity

Time + Space

5

Implement & Test

Example

Need fast indexed access? → Choose **array**

Need frequent insertion/deletion? → Consider **Linked List**

Need LIFO processing? → Choose **Stack**

Need FIFO processing? → Choose **Queue**

Real-World Applications

Arrays

- Student marks
- Image pixels
- Tables
- Sensor data

Linked Lists

- Music playlists
- Browser navigation
- Dynamic memory structures

Stacks

- Undo/Redo
- Function calls
- Browser back button
- Expression evaluation

Queues

- CPU scheduling
- Printer management
- Customer service
- Network requests

Quick Check

Think & Answer

1. Which principle does a stack follow?
2. Which principle does a queue follow?
3. What is the time complexity of accessing an array element by index?
4. Which linked list allows traversal in both directions?
5. What is the Big-O complexity of a loop that runs n times?
6. Why is complexity analysis important?

Thank You

Questions & Discussion

Arrays · Strings · Linked Lists · Stacks · Queues · Complexity